

Leveraged Yield Farming on Blend v2

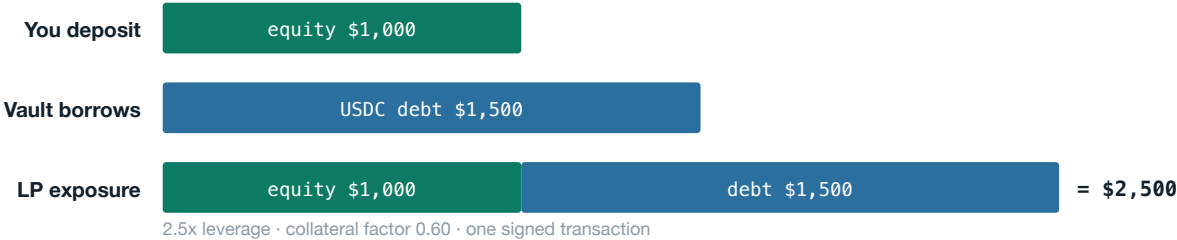
The first leveraged liquidity-provision product on Stellar. A liquidity provider multiplies an Aquarius LP position 2–2.5× in one signed transaction, using the raw LP share token as collateral on Blend. **The engine already works on testnet** — this proposal funds the hardening, audit and mainnet launch.

REQUESTED	CATEGORY	TRANCHES	STATUS
\$106,000	Build	3 · 9 deliverables	Testnet verified

What a user does, in one transaction



One end-of-stack health check — the position is never observably unhealthy. Without the flash loan this is a `supply → borrow → zap → supply` loop that leaves the user liquidatable between passes.



The position. \$1,000 of user equity becomes \$2,500 of LP exposure carrying \$1,500 of USDC debt. The user keeps the upside and the risk on the full \$2,500; the debt is serviced from the LP yield.

Why this is fundable now. All three primitives went live independently: Blend v2 exposes `flash_loan`, Aquarius issues transferable LP share tokens, and Reflector supplies the price feeds the oracle needs. Leveraged LP farming is a mature category on Ethereum and Solana — Pendle on Morpho Blue, Curve LP against crvUSD, Kamino on MarginFi. On Stellar the pieces exist and nobody has connected them.

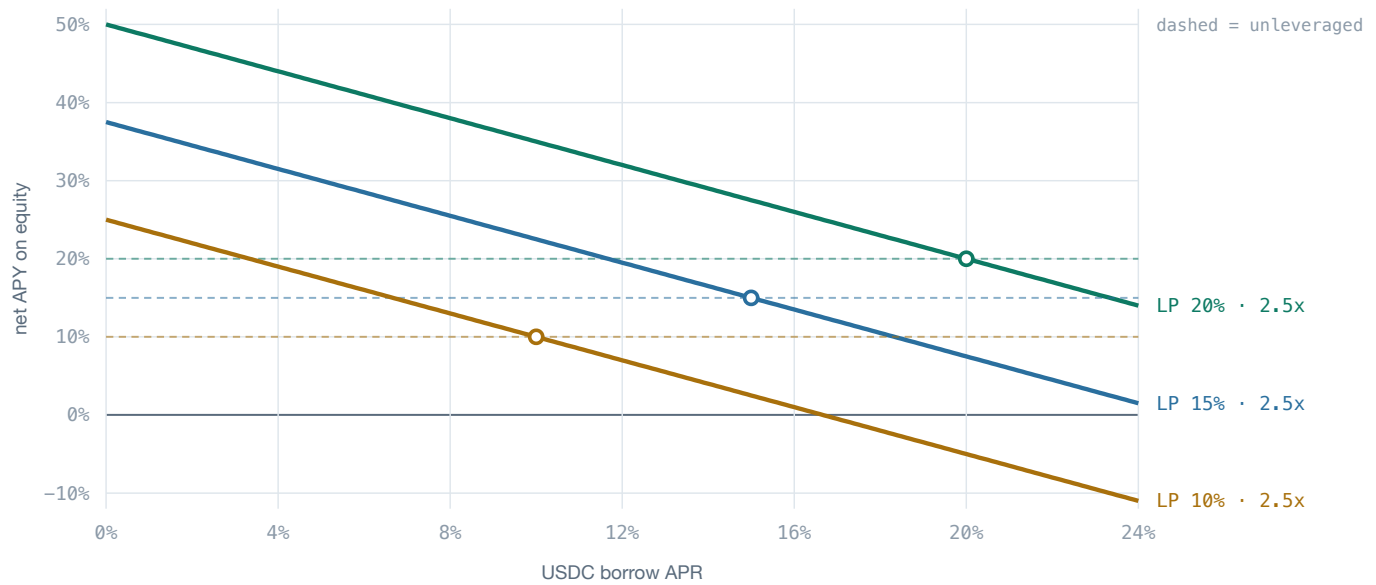
Leverage multiplies a spread — in both directions

We present this as a spread trade, not as “boosted yield”. The arithmetic is simple enough to put in front of a user before they sign.

net APY on equity = (LP APY × L) – (borrow APR × (L – 1))

leverage wins while borrow APR < LP APY

net turns negative at borrow APR > LP APY × L/(L-1) = 1.67 × LP APY at 2.5×



Solid lines: leveraged at 2.5x. Dashed: the same pool unleveraged. Each circle marks where they cross — and the crossing always sits exactly at *borrow APR = LP APY*, whatever the leverage. Left of that point leverage adds return; right of it the borrow cost eats more than the extra exposure earns. Below the zero line the user is paying to farm.

What that means at the launch parameters

LP APY EARNED	BORROW 6%	BORROW 8%	BORROW 10%	BORROW 14%
8%	11.0%	8.0%	5.0%	-1.0%
12%	21.0%	18.0%	15.0%	9.0%
15%	28.5%	25.5%	22.5%	16.5%
20%	41.0%	38.0%	35.0%	29.0%

Stated plainly, because reviewers will find it anyway. Blend borrow rates are utilisation-driven and move without notice. At 2.5x, a 14% borrow rate against an 8% LP yield returns **-1%**. The interface therefore shows the live health factor, the net APY, and the break-even borrow rate for the chosen leverage *before* the user signs — and the risk disclosure requires explicit acknowledgement.

Numbers to confirm before submission. The 20% unleveraged baseline is carried from our Blend one-pager and is not yet independently verified; the 8% borrow figure should be read off live Blend mainnet utilisation. Every return figure above depends on those two inputs.

Raw LP as collateral, priced so it cannot be manipulated

The architecture, and why it changed

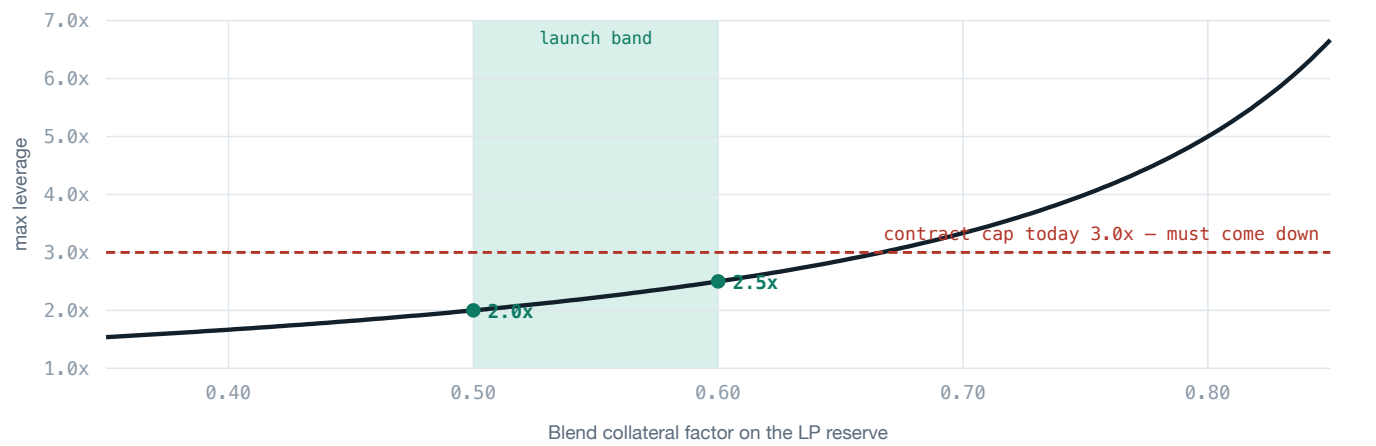
The vault owns the Blend position and tracks each depositor as **shares** of it, so interest and liquidation losses socialise pro-rata with no reconciliation and no off-chain bookkeeping. An earlier design wrapped the Aquarius LP in a 1:1 receipt token and posted the wrapper as collateral. **After architecture review by Script3 — the Blend team — we removed the wrapper entirely** and now supply raw Aquarius LP directly: a shorter liquidator unwind, no intermediate accounting, and one less contract in the audit surface.

Oracle: fair-LP value, not pool spot or TWAP

Pricing LP collateral is the hard part, and pool TWAP alone is unsafe at low TVL — long windows go stale during real moves, short ones invite flash manipulation. We use the fair-value formula adopted by Alpha Homora, Curve and Chainlink reference oracles:

$$LP_price = 2 \cdot \sqrt{(K \cdot P_{xlm} \cdot P_{usdc}) / L} \text{ where } K = x \cdot y \text{ (pool invariant)}$$

Pool state enters **only** through K ; asset prices come from Reflector. An attacker can move reserves cheaply but cannot inflate K beyond what they actually deposit, so the collateral price is not flash-manipulable.



Why leverage is capped at 2.0–2.5x. Maximum leverage is $1/(1-\text{collateral factor})$, so the Blend parameter sets the ceiling. LP collateral is a first-of-kind asset class on Blend, so we proposed a conservative 0.50–0.60 rather than the 0.7+ typical of spot tokens. **The red line is a real finding from preparing this proposal:** the contract's current cap of 3.0x sits above the boundary the launch band implies, and is corrected in Deliverable 2.

Risk posture at launch

PARAMETER	LAUNCH VALUE	RATIONALE
Collateral factor	0.50–0.60	Conservative for a new collateral class
Max leverage	≤ 2.5×	Set at the implied liquidation boundary
Pool TVL cap	\$1M	Bounds ecosystem exposure while the class is proven
Backstop	Treasury	Funded by WhaleHub, not requested from SCF
Liquidation	OSS + bot	Open reference liquidator plus a WhaleHub-run bot from day one

The engine is built and verified — before asking for funding

This is not a proposal to start building. The atomic open, the unwind, liquidation behaviour and the share accounting all work on Stellar testnet today, with transactions on record.

TESTNET LIFECYCLE

Verified

Open, unwind, liquidation, accounting

EXTERNAL REVIEW

Script3

Blend team reviewed the architecture

MAINNET PRODUCT

Live

AQUA staking since Feb 2026

CAPABILITY	EVIDENCE ON TESTNET
Atomic leveraged open	100 LP + 100 borrowed → 149 LP collateral, 100 debt (1.49×)
Partial unwind	repay 50 / withdraw 50 → 99 LP, ~50 debt; 50 LP returned to user
Liquidation	Oracle price drop → Blend auction created and filled with correct terms
Share accounting	Share-based CDP verified against the live Blend position; losses socialise pro-rata
Upgradeability	Vault and zapper both upgradeable behind stable addresses

Three architectural failures found and fixed on testnet

To get there we deployed a complete Blend testnet stack ourselves — pool, oracle, AMM and a self-funded backstop — then hit and fixed three problems that would each have been fatal on mainnet:

- **Soroban re-entrancy.** The vault was its own flash-loan receiver, so Blend's callback re-entered a contract already on the stack. Fixed by splitting the zap into a separate contract.
- **Share-model desync under liquidation.** An internal ledger drifted from the real Blend position once a liquidation touched it. Fixed by deriving every position from live Blend state instead of a stored balance.
- **Authorisation gap in the non-flash unwind.** Blend pulls repayment via a transfer that needed explicit contract authorisation. Fixed and verified end-to-end.

Finding these on testnet, against a stack we funded ourselves, is the argument for funding the remaining work: the expensive discovery has already happened.

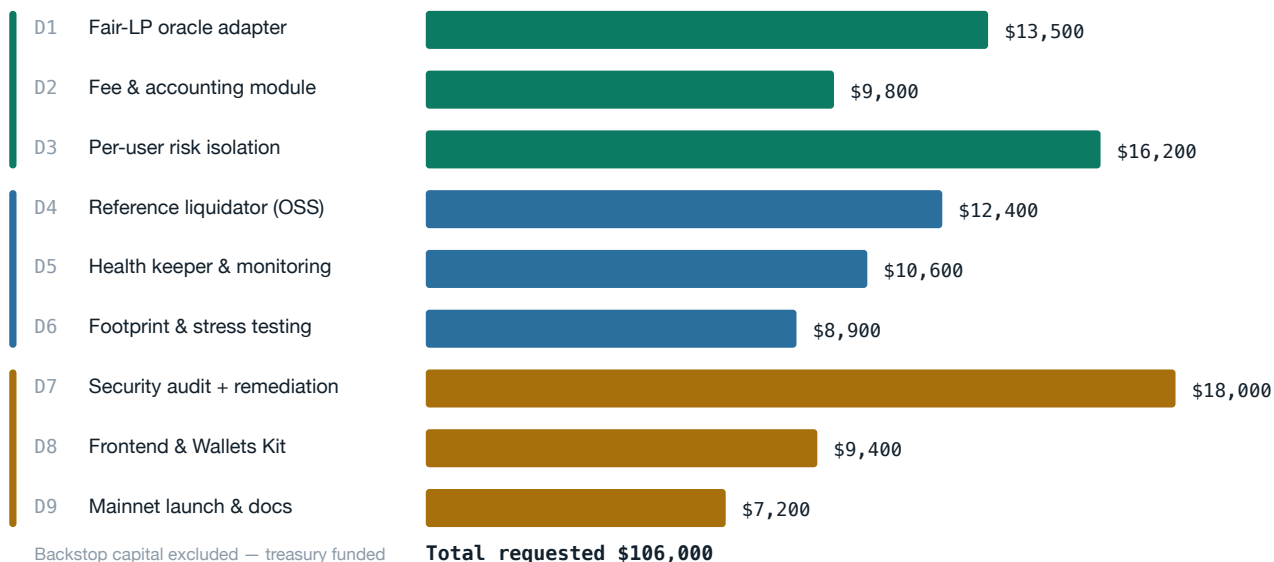
Operating history

WhaleHub has run a live mainnet product since February 2026 — AQUA staking with auto-compounding Aquarius rewards at `app.whalehub.io` — alongside a 25-article public research library. We ship and maintain production Soroban contracts under a multisig-governed upgrade process.

On the choice of pair. We are deliberately not building this on our own BLUB–AQUA pool: it sits outside the Aquarius gauge whitelist and is too shallow to lever safely. The product targets the largest real Aquarius pairs, starting with XLM/USDC — which is also where the ecosystem benefit is, since each leveraged position deposits 2–2.5× the LP a user could provide alone.

\$106,000 across three tranches

Sequenced so the economics and risk core land first, safety infrastructure second, and audit plus mainnet last. Backstop capital is excluded from the ask.



Audit is the largest single line and non-negotiable: this contract composes user funds with a third-party lending protocol. The reference liquidator (D4) is built as MIT-licensed ecosystem infrastructure — any future LP-collateral market on Blend can reuse it.

#	DELIVERABLE	COMPLETES	BUDGET
D1	Fair-LP oracle adapter. Invariant-based Reflector pricing, deviation breakers, feed-outage fallback, flash-manipulation test suite.	15.09.26	\$13,500
D2	Performance-fee and accounting module. Fee on net profit after borrow costs; high-water mark per share. Corrects the 3.0x cap to $\leq 2.5\times$.	29.09.26	\$9,800
D3	Per-user risk isolation. Per-user health and liquidation, so one over-levered depositor cannot impose losses on the rest.	13.10.26	\$16,200
D4	Open-source reference liquidator. MIT-licensed four-step atomic LP unwind, with docs and a testnet dry-run harness.	27.10.26	\$12,400
D5	Health keeper and monitoring. Tiered user alerts, a WhaleHub-run liquidation bot from day one, public risk dashboards.	10.11.26	\$10,600
D6	Footprint validation and stress testing. Liquidation sequence measured against Soroban resource limits; crash, rate-spike and bank-run scenarios.	24.11.26	\$8,900
D7	Third-party security audit and remediation. Vault, zapper, oracle and liquidator, by a Soroban-experienced firm. Published report.	08.12.26	\$18,000
D8	Production frontend and Wallets Kit. Live zap quoting, health and break-even shown pre-signature, hardware-wallet support, risk disclosure.	22.12.26	\$9,400
D9	Mainnet launch and documentation. Pool deployment with treasury backstop, staged launch under the \$1M cap, public integration guide.	12.01.27	\$7,200

What the ecosystem gets, beyond WhaleHub

Three of the four outputs below are reusable by teams that are not us. That is deliberate — it is the argument for community funding rather than private capital.

OUTPUT	WHO BENEFITS
A new collateral class on Blend	LP tokens become productive collateral, with the oracle design and parameter work published for reuse.
An MIT-licensed reference liquidator	Any future LP-collateral market on Blend can fork D4 rather than rebuild the four-step unwind.
Deeper Aquarius liquidity	Each leveraged position deposits 2–2.5× the LP a user could supply alone.
Documented failure modes	The re-entrancy, share-desync and authorisation bugs we hit are published so others avoid them.

Team

Viktor Vostrikov — Founder. Builds and operates WhaleHub: the Soroban staking contract, the auto-compounding vault system, the backend reward indexer, and the mainnet multisig upgrade process. Has shipped and maintained production Soroban contracts holding real user funds since early 2026. Led the Blend v2 integration through testnet, including the self-funded backstop deployment and the Script3 architecture review.
Contact viktor@whalehub.io

Sources and assumptions

INPUT	VALUE USED	SOURCE
Collateral factor	0.50–0.60	Proposed to Script3; under discussion
Max leverage	2.5×	Derived: 1/(1–0.60)
Contract cap today	3.0×	Read from the vault source (max_leverage_bps)
Unleveraged LP APY	20%	Carried from our Blend one-pager — to verify against live Aquarius XLM/USDC
USDC borrow APR	8%	Assumption — to read off live Blend mainnet utilisation
Testnet position figures	1.49×	Verified on-chain, Stellar testnet

Testnet contracts. Vault `CC2JF2VP3LVNHYI7URF3R376FCVFSAI4HNVZ2WPZZHBJDAJSWX2X2PFH` · Zapper `CCUDFI62LH2IMHGSRBLLF7SIKRPDQ3LZPA4TYFBFS5ENRYWZ0HOKSHT2` · working UI at `app.whalehub.io/leverage` (testnet-pinned).

Not investment advice. All APY figures are arithmetic outputs of the stated assumptions, not projections or promises of return. Leveraged positions can be liquidated and can lose more than an unleveraged position. BLUB is a floating asset and is not pegged or redeemable.